For the IRIS dataset that has been shared previosuly- Implement the KNN algorithm.

Report the classification accuracy for K= 1, 3, 5.

Use the distance metric- Euclidean and City block distance

Plot the confusion matrix for all the values of K and report the accuracy for both the distance measures

Need to evaluate KNN under 2 variables:

Variable 1: K

```
K = 1, 3, 5
```

Variable 2: Distance metric

```
Euclidean distance

City block (Manhattan) distance
```

Helper Functions

In [1]:
```
import numpy as np
from collections import Counter
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

In [5]:
```
def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1-x2)**2))


def manhattan_distance(x1, x2):
    return np.sum(np.abs(x1-x2))

def accuracy(y_true, y_pred):
    correct = np.sum(np.array(y_pred)==np.array(y_true))
    return correct / len(y_true)

def confusion_matrix(y_true, y_pred):
    classes = sorted(set(y_true))
    n = len(classes)

    matrix = np.zeros((n,n), dtype=int)

    for i in range(len(y_true)):
        actual = y_true[i]
```

```
        predicted = y_pred[i]
        matrix[actual][predicted] += 1

    return matrix


def normalize(X):
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)

    return (X-mean) / std

def knn_predict(X_train, y_train, X_test, k, distance_fn):
    predictions = []

    for test_point in X_test:
        distances = []

        for i in range(len(X_train)):
            dist = distance_fn(test_point, X_train[i])
            distances.append((dist, y_train[i]))

        distances.sort(key=lambda x:x[0])
        k_neignbors = distances[:k]
        k_labels = [label for a, label in k_neignbors]

        most_common = Counter(k_labels).most_common(1)[0][0]
        predictions.append(most_common)
    return predictions
```

Main

In [6]:
```
def main():

    data = load_iris()
    X = data.data
    y = data.target

    X = normalize(X)

    X_train, X_test, y_train, y_test = train_test_split(X, y, random_stat

    for distance_metric in [euclidean_distance, manhattan_distance]:
        print(f"Distance Mettric : {distance_metric.__name__}")

        for k in [1,3,5]:

            predictions = knn_predict(X_train, y_train, X_test, k, distan
            acc = accuracy(y_test, predictions)
            cf_matrix = confusion_matrix(y_test, predictions)

            print(f"\nK = {k}")
            print(f"Accuracy: {acc:.4f}")
            print("Confusion Matrix:")
            print(cf_matrix)
```

In [7]:
```
main()
```

```
Distance Mettric : euclidean_distance

K = 1
Accuracy: 0.9474
Confusion Matrix:
[[14  0  0]
 [ 0 10  1]
 [ 0  1 12]]

K = 3
Accuracy: 0.9474
Confusion Matrix:
[[14  0  0]
 [ 0 10  1]
 [ 0  1 12]]

K = 5
Accuracy: 0.9474
Confusion Matrix:
[[14  0  0]
 [ 0 10  1]
 [ 0  1 12]]
Distance Mettric : manhattan_distance

K = 1
Accuracy: 0.8684
Confusion Matrix:
[[14  0  0]
 [ 0  9  2]
 [ 0  3 10]]

K = 3
Accuracy: 0.9474
Confusion Matrix:
[[14  0  0]
 [ 0 10  1]
 [ 0  1 12]]

K = 5
Accuracy: 0.9474
Confusion Matrix:
[[14  0  0]
 [ 0 10  1]
 [ 0  1 12]]
```